

HPCC: High Precision Congestion Control

Rui Miao

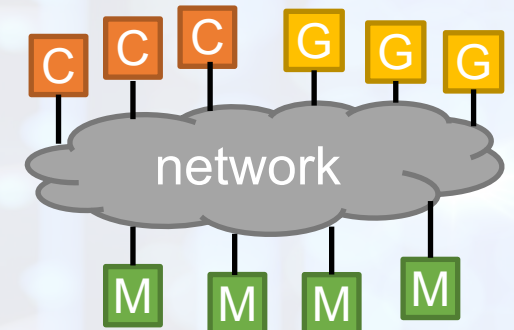
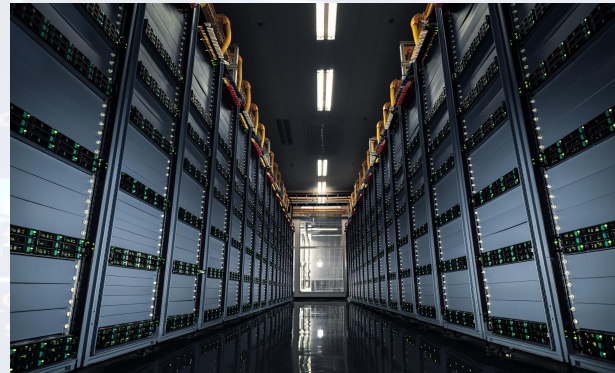
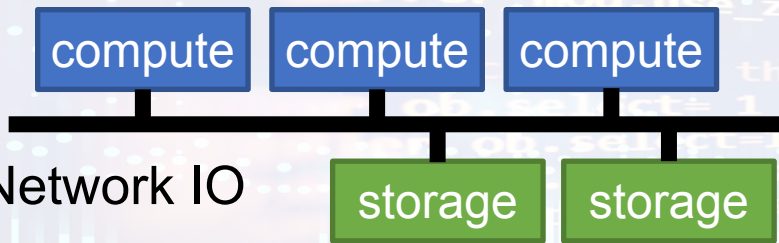
Yuliang Li, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao,
Ming Zhang, Frank Kelly, Mohammad Alizadeh, Minlan Yu

High-performance networking is desired

High-performance storage

Distributed deep learning

Resource disaggregation



- HDD □ SSD □ NVMe
- Higher-throughput, lower latency
- 1M IOPS / 50~100us

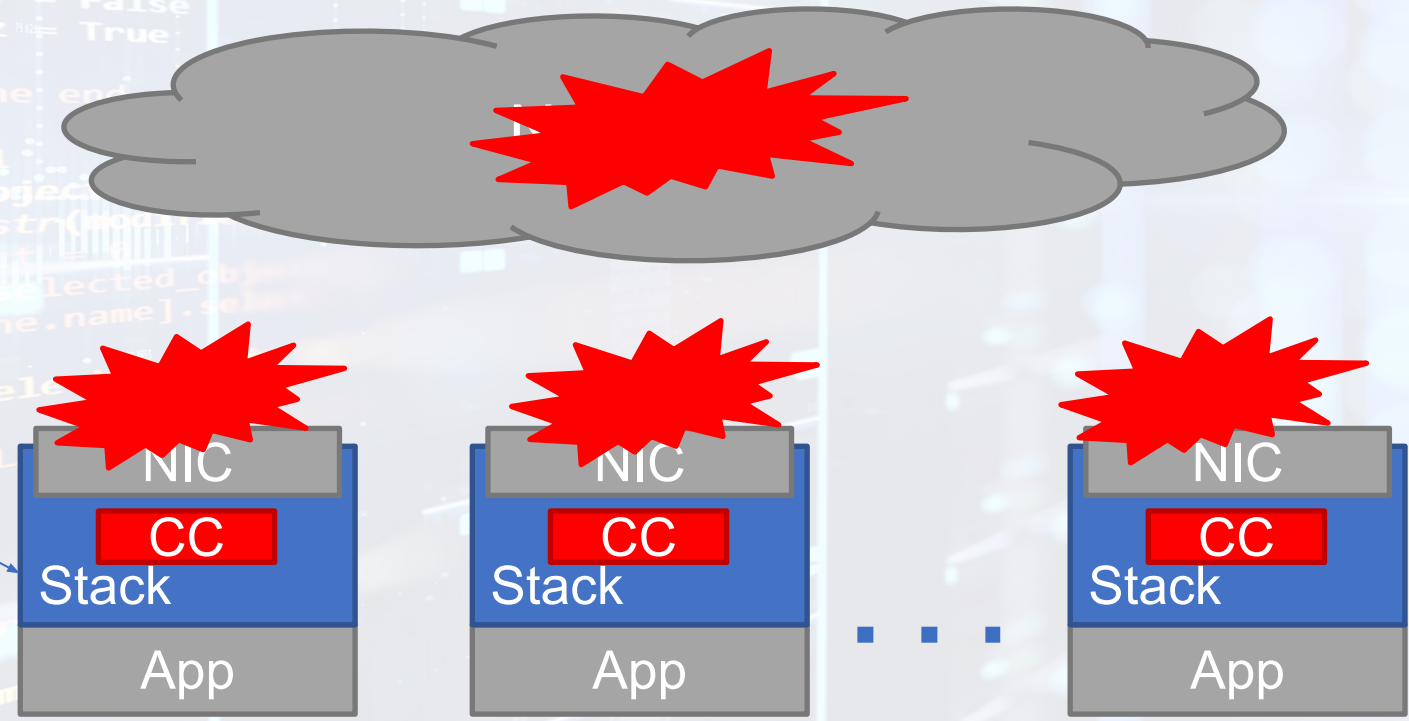
- CPU □ GPU, FPGA, ASIC
- Faster compute, lower latency
- E.g. latency <10us

- More network load
- Need ultra-lower latency: 3-5us

Two essential steps to high performance

More severe congestion
So Congestion control (CC) is very important!

Many ongoing efforts
on hardware offloading
- RDMA, SmartNIC



CC is immature in high-speed network

- State-of-the-art CC in hardware-offloading solutions
 - DCQCN^[SIGCOMM 15]: ECN-based
 - TIMELY^[SIGCOMM 15]: delay-based
- Face problems in operating large-scale RDMA networks
- Root cause is the CC

Problems of state-of-the-art

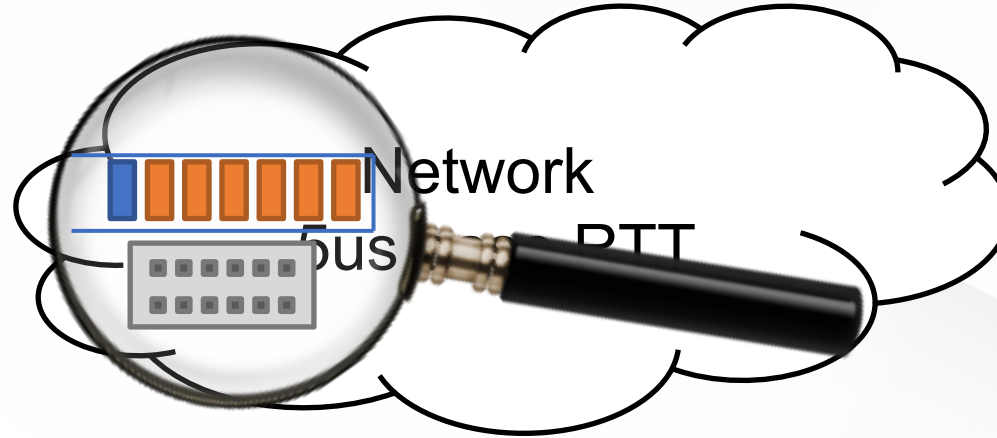
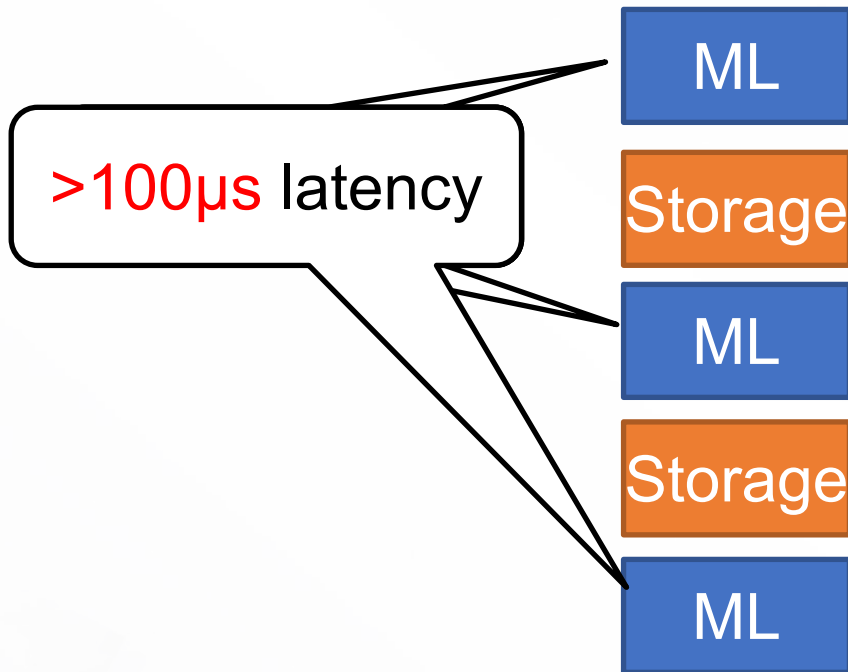
- Stability issues: incast/failure cause PFC storms or deadlock
- Slow convergence
- Challenge in reconciling BW-hungry app and latency-sensitive app
- Standing queue
- Parameter tuning takes months
- CC parameters is hard to tune

Problem 1: slow convergence

- Operators' problem
 - Buffer overflow happens during incast/failure
 - ▶ Use PFC to prevent loss: good performance on average, threat from PFC storm and deadlock
 - ▶ Disable PFC: bad performance on average.
- Root cause: CC cannot control the buffer well, because of slow convergence
 - Imprecise feedback (ECN, delay) cannot tell the rate mismatch
 - CC resolves congestion slowly, by halving the rate per RTT
 - More severe in higher speed networks, because filling buffer faster
- Ideally, adjust to the right rate in one RTT
 - So loss is rare, and do not need PFC

Problem 2: standing queues

- Operators' problem:
 - Hard to run BW-hungry app and latency-sensitive app in the same cluster



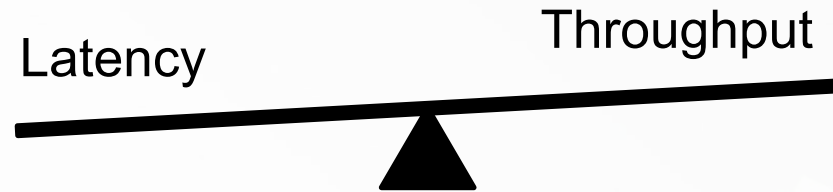
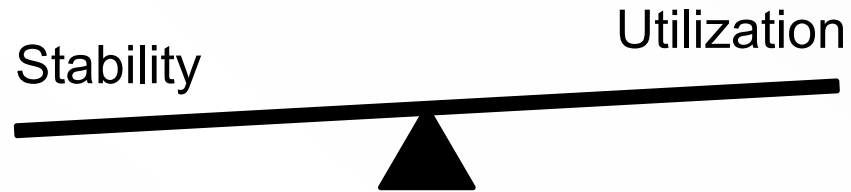
Problem 2: standing queues

- Root cause is CC
- Feedback (ECN/delay) relies on the queue
- CC intentionally keeps standing queues

Add 20~50us queueing delay
4~10x base RTT!!!

Problem 3: complex parameter tuning

- Tradeoffs



- Many factors affect the tradeoffs
 - Traffic patterns, failure scenarios, and network arch
- Feedback (ECN/delay) is imprecise
 - CC has to use heuristics to guess (1) network condition; (2) rate adjustments
- Many parameters in such heuristics
 - e.g., 15 parameters in DCQCN
 - Tuning takes several months

Problems of state-of-the-art

- One fundamental issue: **coarse-grained feedback**
 - Slow convergence
- No precise **feedback** indicating how much to increase/decrease
 - Standing queue
- **Feedback** relies on queue
 - Complex parameter tuning
- No precise **feedback** ☐ need heuristics: lots of parameters

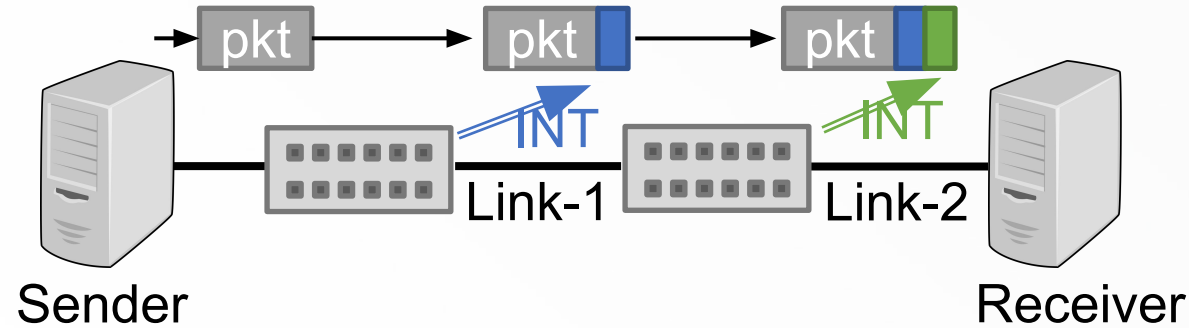


What if we have precise feedback?



HPCC: use INT as precise feedback

- In-band network telemetry (INT) provides many details per packet



- Broadcom&Barefoot have INT in recent products.
- Widely used for diagnosis and monitoring in production

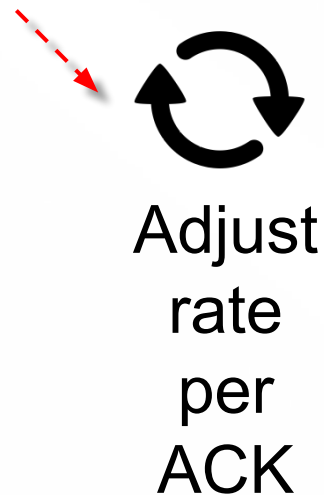


How good can CC do with INT as the feedback?
We design HPCC to answer this question

HPCC solves the 3 problems

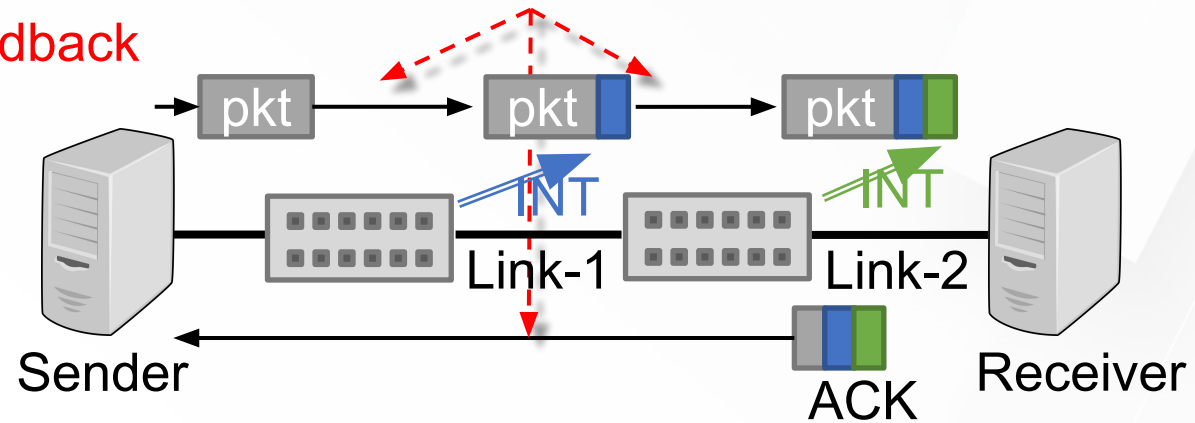
- ❑ Using INT as the precise feedback
 - Fast convergence
- ❑ Sender knows the precise rate to adjust to, on every ACK
 - Near-zero queue
- ❑ Feedback does not rely on queue
 - Few parameters
- ❑ Precise feedback, so no need for heuristics which requires many parameters

Challenge 2: Overreaction:
- Diff ACKs bring overlapping feedback

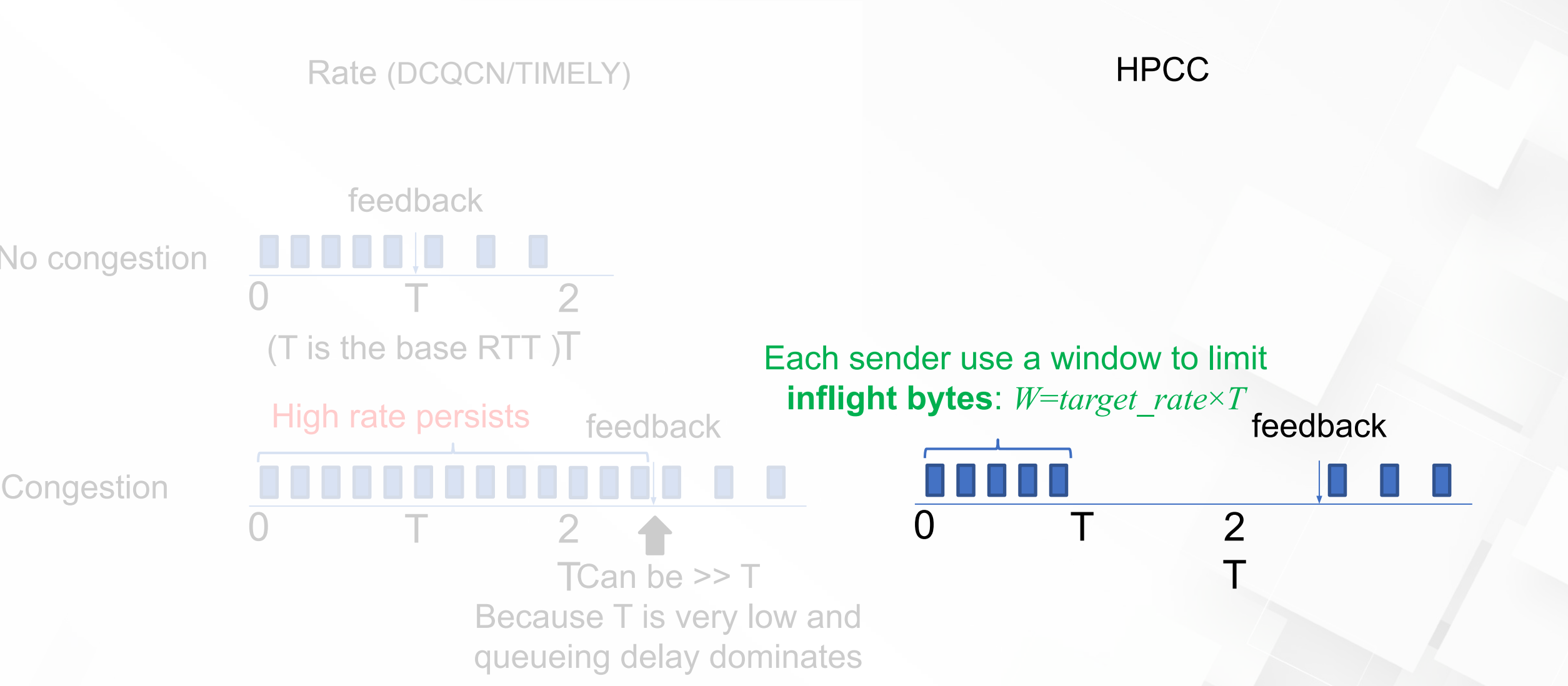


Challenge 1: Feedback delay:

- Pkt/ACK may gets delayed



Challenge 1: tolerate feedback delay



Challenge 1: tolerate feedback delay

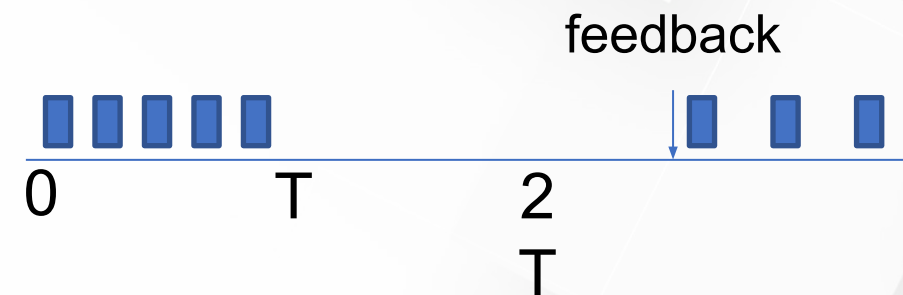
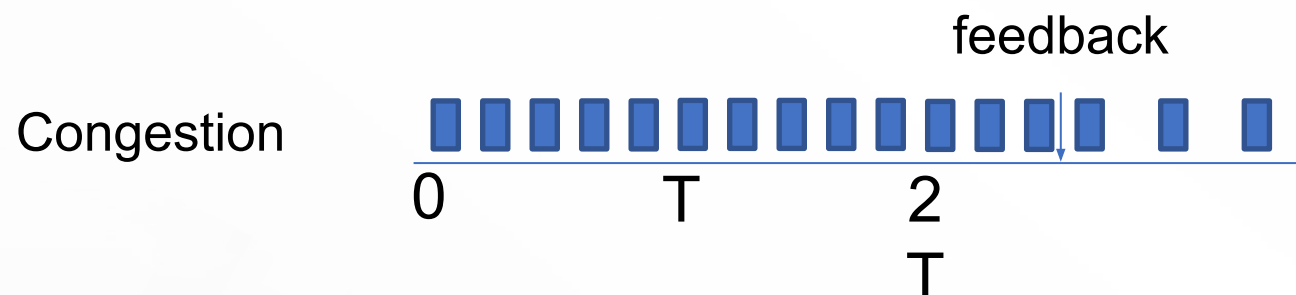
Need a new way of measuring congestion:

Rate-based scheme

- Use rate mismatch

HPCC

- Rate mismatch is bad measurement
- Actual sending rate is non-constant, due to the inflight bytes limit



Estimating inflight bytes

- Use **total inflight bytes** to measure congestion
 - Each flow needs to estimate the total inflight bytes
 - Use INT to estimate the total inflight bytes



$$\text{total inflight bytes} \approx \underbrace{qlen + txRate \times T}_{\text{Available in INT}}$$

Each sender estimates independently in a distributed way

Controlling inflight bytes

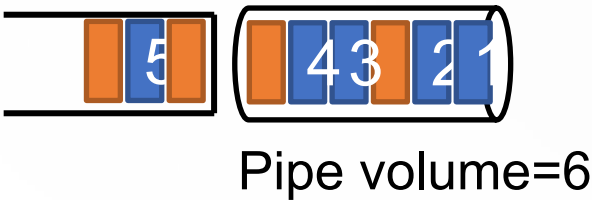
- Use **total inflight bytes** to measure congestion
 - Each flow needs to estimate the total inflight bytes
 - Use INT to estimate the total inflight bytes
- Compute adjustment based inflight bytes
 - Use MIMD to quickly adjust to the right rate

Challenge 2: Fast reaction without overreaction

Per-ACK reaction

- Overreaction

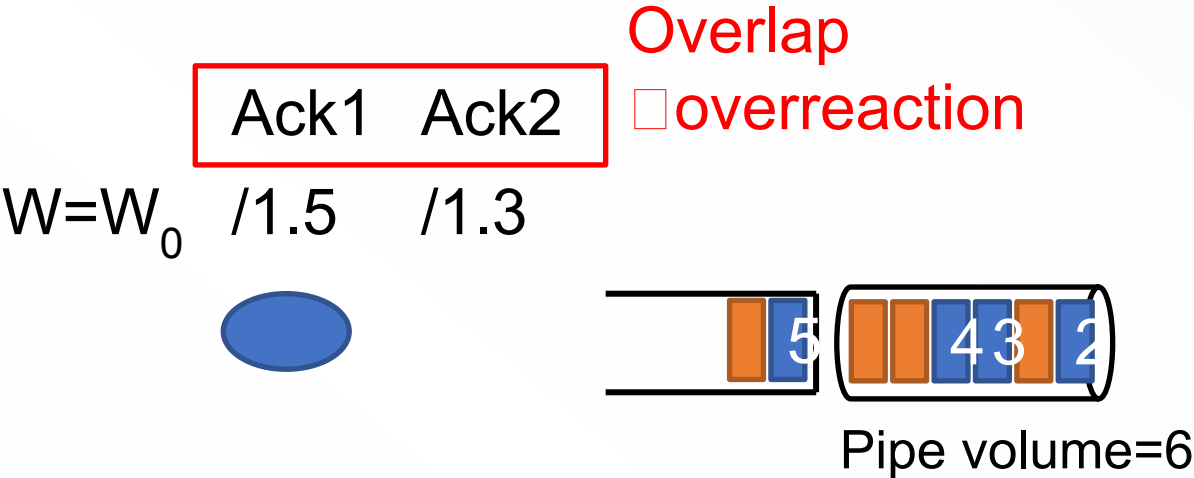
Ack1
 $W=W_0 / 1.5$

Challenge 2: Fast reaction without overreaction

Per-ACK reaction

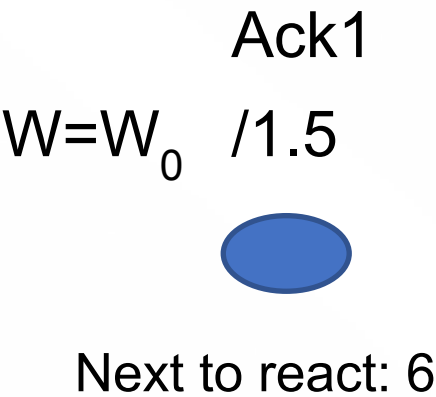
- Overreaction



Challenge 2: Fast reaction without overreaction

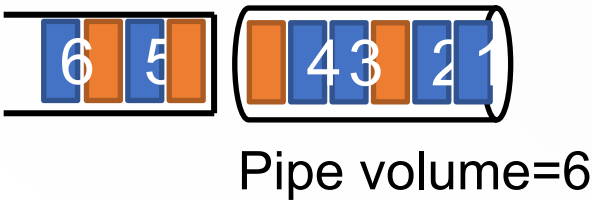
Per-ACK reaction

- Overreaction



Per-RTT reaction

- No overreaction



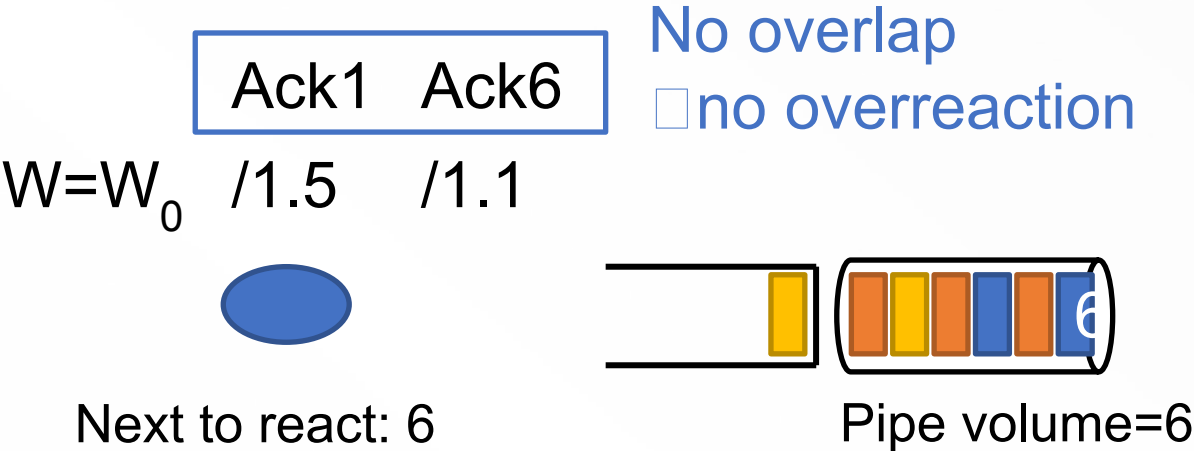
Challenge 2: Fast reaction without overreaction

Per-ACK reaction

- Overreaction

Per-RTT reaction

- No overreaction



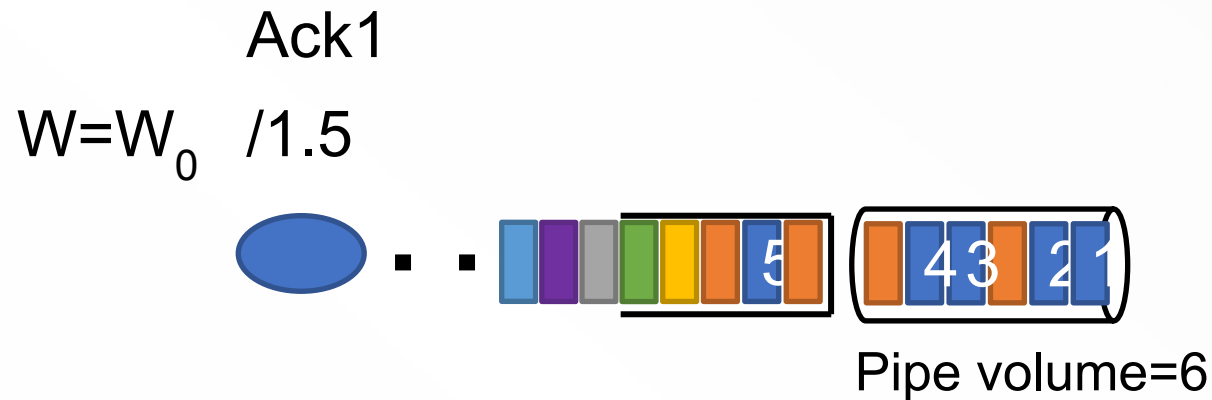
Challenge 2: Fast reaction without overreaction

Per-ACK reaction

- Overreaction
- Fast reaction to sudden bursts

Per-RTT reaction

- No overreaction
- Miss sudden bursts



Challenge 2: Fast reaction without overreaction

Per-ACK reaction

- Overreaction
- Fast reaction to sudden bursts

Per-RTT reaction

- No overreaction
- Miss sudden bursts

HPCC combines per-ACK and per-RTT (details in paper)

- No overreaction
- Fast reaction to sudden bursts

- Asynchronized flows
- Arbitrary topology

Convergence speed

$$Y(n) = AR(n)$$

$$R_j(n+1) = \frac{R_j(n)}{\max_i \{Y_i(n)A_{ij}/C_i\}}$$

Converge to Pareto optimal

Queueing delay

$$(\pi N/8)^{0.5}$$

Queue size is bounded

Fairness

$$R_i = a \left(1 - \frac{U_{target}}{U_i} \right)^{-1}$$

Max-min/proportional fairness

Implementation

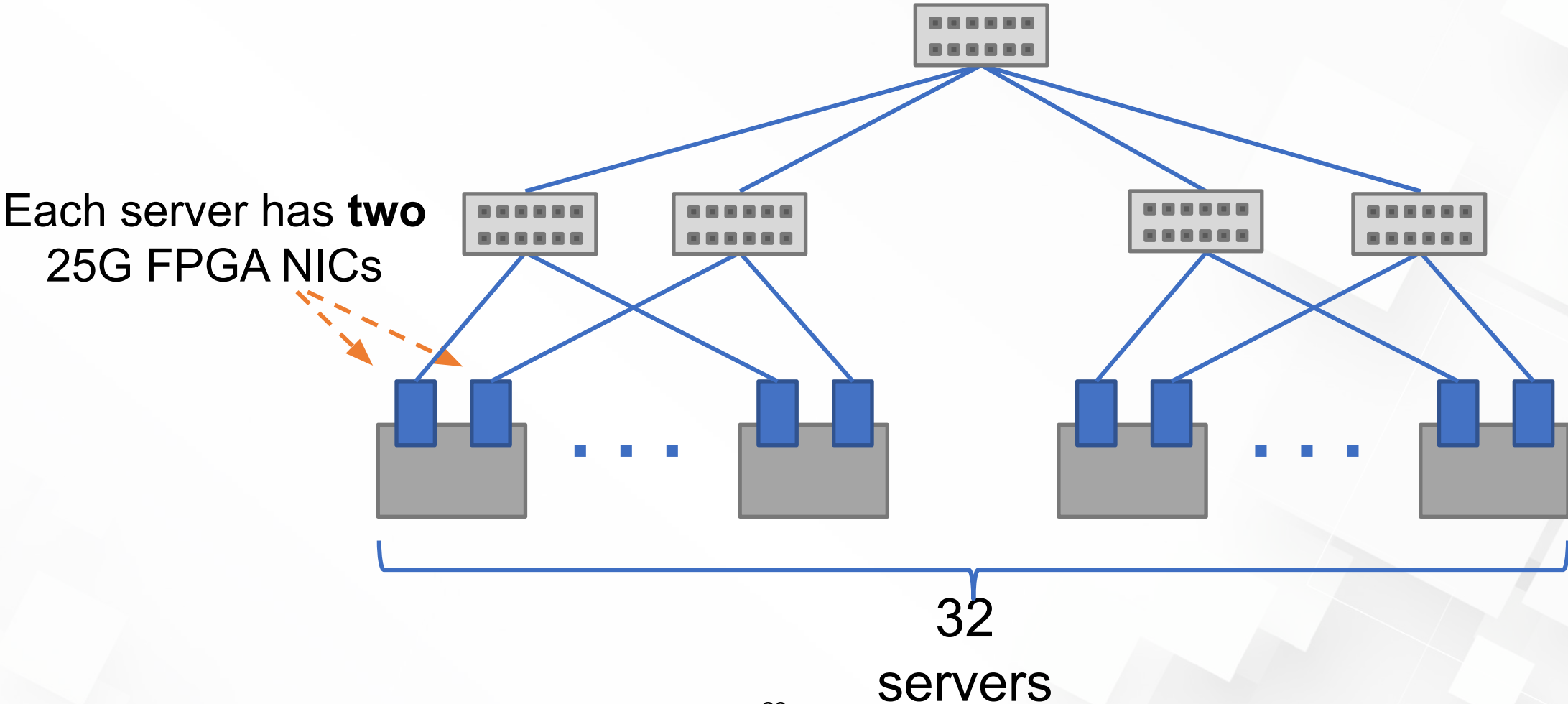
- Commodity FPGA NIC
- In a 32-server testbed

Implementation in commodity FPGA NIC

- 14,000 lines of Verilog code for the whole NIC
- 2,000 lines for CC
 - <2% of hardware resources
- Optimization:
 - look-up table: 8x speedup for division
 - h/w friendly compression with <1% error: 10KB for [1, 4M]
 - Multiple parallel engines: 6x more concurrent flows
- INT header overhead: 4.2% (MTU=1KB)



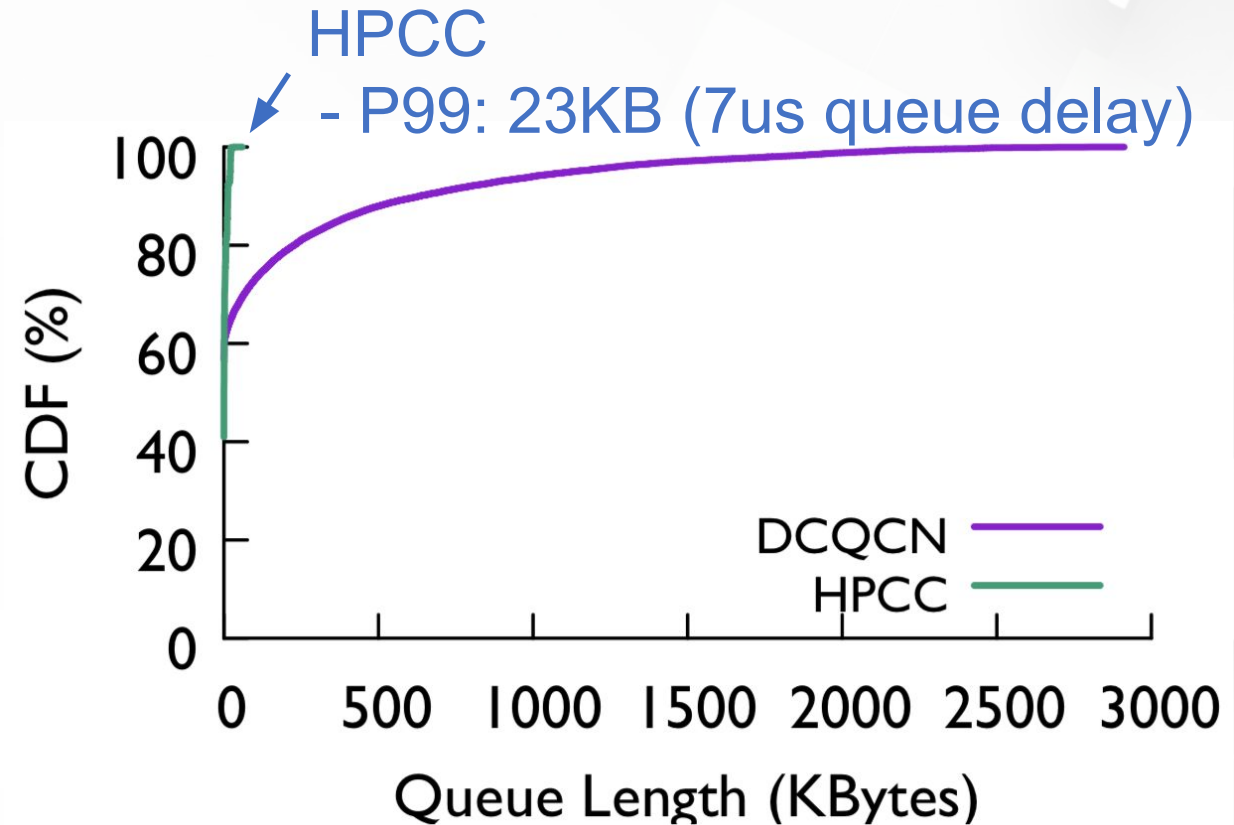
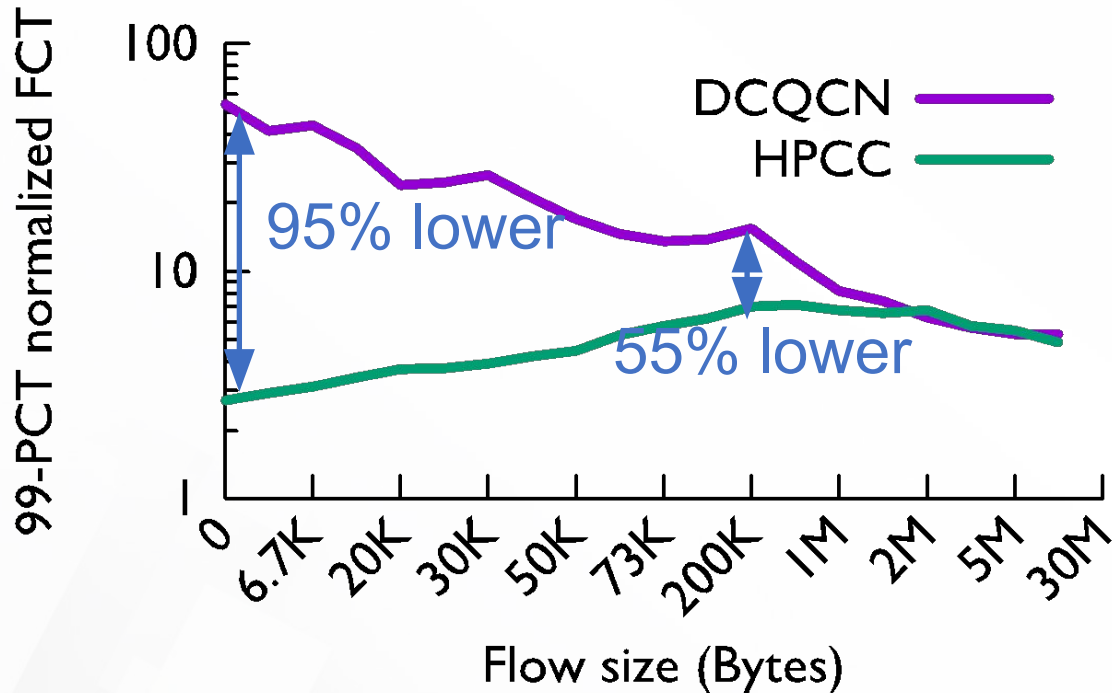
Implementation: testbed



- Setup
 - 32-server, 5-switch Testbed (25GE)
 - 320-server, 56-switch simulation, (100GE)
- Traffic setting
 - Web search, with diff levels of traffic load
 - Hadoop, with diff levels of traffic load
 - Diff incast sizes & ratios
 - Many micro-benchmarks
- Compare with DCQCN, TIMELY, DCTCP, with their recommended parameters
- Metrics
 - Flow completion time (FCT)
 - Queue length distribution

HPCC achieves lower FCT and near-zero queue

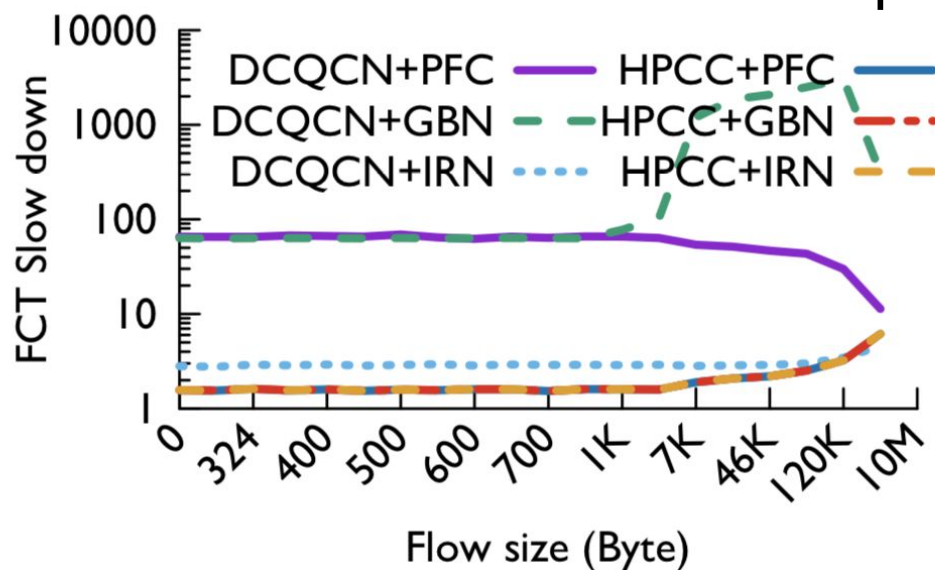
- In testbed, vs. DCQCN (hardware-based, widely used in industry)
 - Web search traffic at 50% load
- Vs. other CC (unavailable in HW) in simulation. HPCC performs better



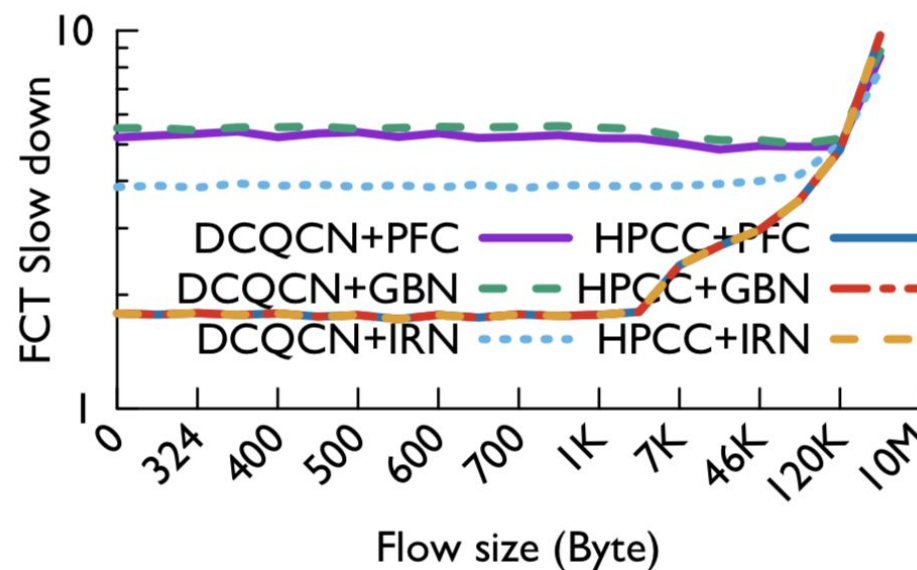
HPCC has the potential to remove PFC

- HPCC is persistently better, regardless of the flow control schemes
- HPCC is insensitive to the flow control schemes

95 percentile FCT



(a) 30% Avg. Load + incast



(b) 50% Avg. Load

Practical impact

- Demonstrate problems of existing CC with real production experience
- Implement HPCC with HW NIC & switch

Advance in CC

- Demonstrate using precise info from INT make CC highly performant
- Address the challenge of using INT
- Proof for convergence speed, queueing delay, and fairness

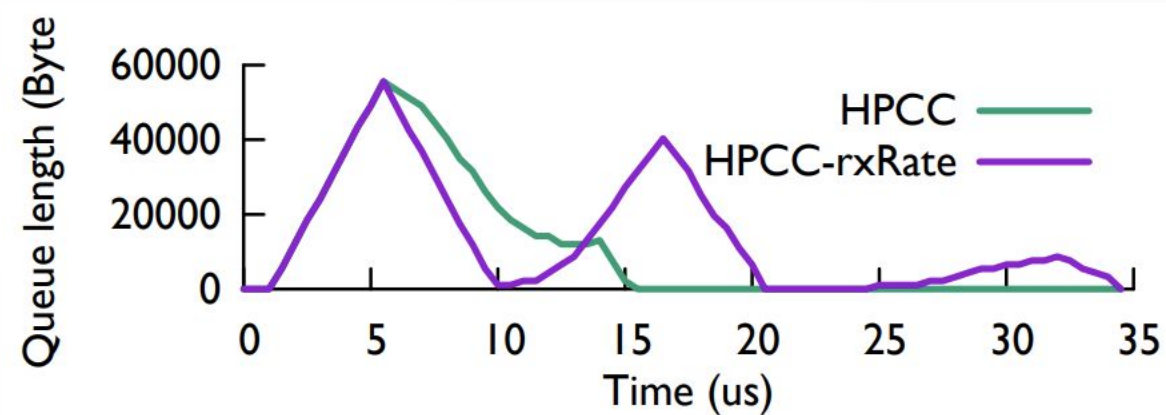
Collaborating with switch and NIC partners on deploying HPCC in Alibaba's production networks

Difference with XCP

- XCP adjust rate heuristically
 - $\alpha \cdot rxRate \cdot d + \beta \cdot qlen \leftarrow$ no clear physical meaning
- HPCC
 - $txRate \cdot T + qlen \leftarrow$ total inflight bytes (clear physical meaning)

txRate vs rxRate

- *rxRate* is a overlapping term with *qlen*
 - A high *rxRate* increases the queue occupancy □ large *qlen*



Multiple QoS queues?

- Queue is a scarce resource in the cloud
 - App has higher priority in using the queues (than network operator)
 - Management/control plane traffic
 - Diff CC mechanism
- HPCC is complementary to using multiple-queue